



PARAMVIR SINGH

AGENDA

- Patterns of the day
 - Singleton
 - Immutable
 - Factory

SINGLETON - DESCRIPTION

- **Context:**
 - It is very common to find classes for which only one instance should exist (*singleton*)
- **Problem:**
 - How do you ensure that it is never possible to create more than one instance of a singleton class?
- **Forces:**
 - The use of a public constructor cannot guarantee that no more than one instance will be created.
 - The singleton instance must also be accessible to all classes that require it.
- **References:**
 - E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, October 1994.

ONE POSSIBLE SOLUTION

- Create a class defining:
 - A private constructor
 - A *non-private* getter method to:
 - return the only (*universal*) object, or otherwise
 - create one by calling the constructor
 - The getter is responsible for creating the universal object
 - **Can you think of alternate designs?**

«Singleton»
<u>theInstance</u>
<u>getInstance()</u>

Company
<u>theCompany</u>
Company() «private» <u>getInstance()</u> — — —

```
if (theCompany==null)
    theCompany= new Company();

return theCompany;
```

EXAMPLE CODE

- Librarian code (Canvas)

SINGLETON BUILT-IN EXAMPLES

***Are there any
singletons in
Java API library??***

SINGLETON BUILT-IN EXAMPLES

***Are there any
singletons in
Java API library??***

java.lang.Runtime#getRuntime()

java.awt.Desktop#getDesktop()

java.awt.Toolkit#getToolkit()

THE IMMUTABLE PATTERN

- **Context:**
 - An immutable object is an object that has a state that never changes after creation
- **Problem:**
 - How do you create a class whose instances are immutable?
- **Forces:**
 - There must be no loopholes that would allow 'illegal' modification of an immutable object
- **References**
 - M. Grand, *Patterns in Java Volume 1: A Catalog of Reusable Design Patterns Illustrated with UML*, 2nd edition, Wiley, 2002

ONE POSSIBLE SOLUTION

- Ensure that the **constructor** of the immutable class is the ***only*** place where the values of instance variables are set or modified.
- **Instance methods** which access properties must not have side effects.
- If a method that would otherwise **modify an instance variable is required**, then it has to return a new instance of the class.

CONCRETE STEPS FOR SOLUTION

One way to implement the general solution:

- Declare the class as **final** so it can't be extended.
- Make all **fields private** so that direct access is not allowed.
- Make all **fields final** so that it's value can be assigned only once.
- **Don't provide setter methods** for variables

EXAMPLE CODE

-
- Book code (Canvas)

IMMUTABLE BUILT-IN EXAMPLES

***Are there any
immutables in
Java API library??***

IMMUTABLE BUILT-IN EXAMPLES

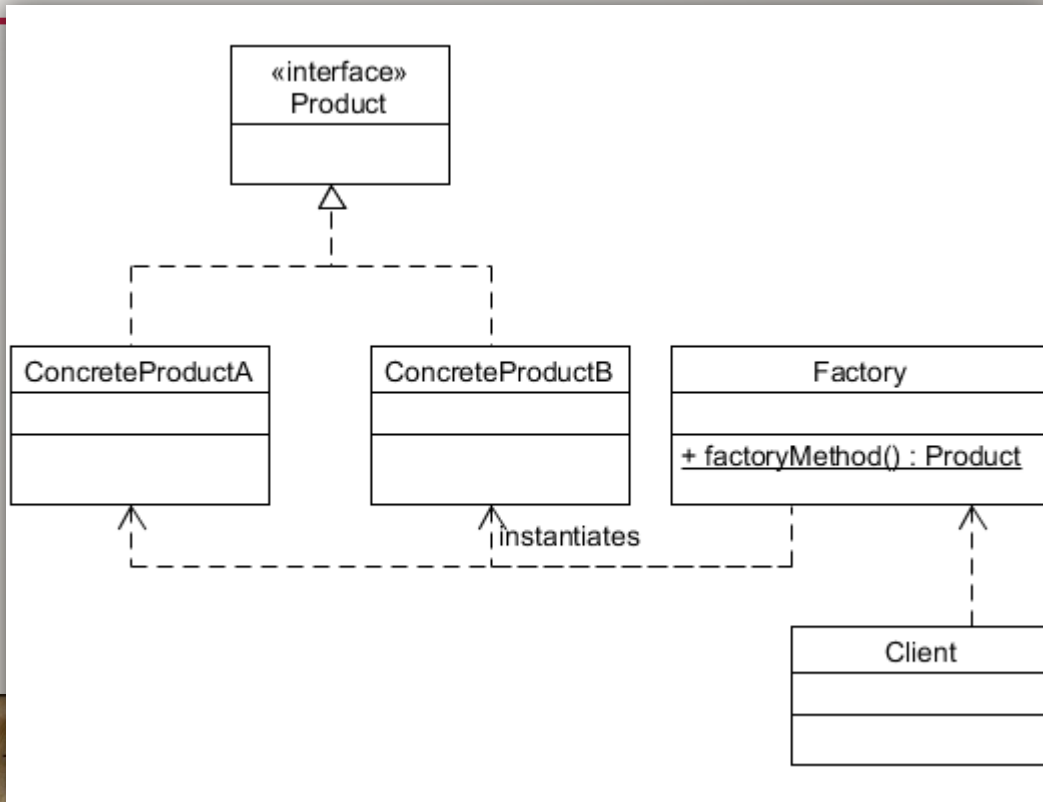
***Are there any
immutables in
Java API library??***

java.lang.String

java.lang.Integer

THE FACTORY METHOD PATTERN

- The **Factory Method** pattern is an established solution to the kind of problem where you know **when** to create an object – but you don't know **what** class to instantiate
- Key consequences of the pattern:
 - Programming to an interface rather than an implementation is encouraged (OCP/DIP)
 - => flexibility, robustness
 - The pattern provides a mechanism that allows clients to request new objects without specifying their concrete types



THE FACTORY METHOD PATTERN - EXAMPLE

```
public class ShapeFactory {  
    //use getShape method to get object of type shape  
    public Shape getShape(String shapeType){  
        if(shapeType == null){  
            return null;  
        }  
        if(shapeType.equalsIgnoreCase("CIRCLE")){  
            return new Circle();  
        } else if(shapeType.equalsIgnoreCase("RECTANGLE")){  
            return new Rectangle();  
        } else if(shapeType.equalsIgnoreCase("SQUARE")){  
            return new Square();  
        } return null;  
    }  
}
```

EXAMPLE CODE

- VehicleFactory code (Canvas)

SUGGESTED TASKS

- Write classes that are:
 - Singleton and Immutable
 - Singleton and Factory
- Are there any factory classes in Java API library?

REVIEW

Singleton, immutable and factory are three kinds of object creation patterns.

Singleton pattern

- Make sure only one object is created for this class
 - need to amend the class design to implement this
- Other classes should be able to access that only object

Immutable pattern

- Make sure that an object once create and initialized, must not allow any change in its state (values of its fields)
 - need to amend the class design to implement this
- If the fields are of other class types, those classes must also be defined to be immutable, especially if the references to these objects are directly accessible outside the immutable class

Factory pattern

- Create a class that has the responsibility of creating the desired object type for a super type
- Use switch or if-else structures to handle creation of all or some subclass types

REFERENCES

- *Books:*
 - Object-Oriented Software Engineering: Practical Software Development using UML and Java by Timothy Lethbridge and Robert Laganieri, Chapter 6
 - Design Patterns: Elements of Reusable Object-Oriented Software by John Vlissides, Ralph Johnson, Richard Helm, Erich Gamma
- *Other:*
 - Robert Martin, Design Principles and Design Patterns, https://fi.ort.edu.uy/innovaportal/file/2032/1/design_principles.pdf